st5-guide

System Prompt for the LLM:

> *"You are an Expert Data Visualisation Engineer. Write a Python script for 'Step 4: Visualisation & Evaluation' of an emergency tracking pipeline. The script must read three CSVs (predictions, features, cluster info) and output an 8-page Dark Theme PDF report using `matplotlib` / `seaborn`, and a 4-layer interactive HTML map using `folium`. Follow these exact instructions to ensure the formatting and logic match the specifications perfectly."*

🎨 1. Setup, Styling, & Math

Instruct the LLM to set up the environment and aesthetic constants:

- **Libraries:** `pandas`, `numpy`, `matplotlib` (set backend to `"Agg"`), `seaborn`, `folium`, and `folium.plugins.HeatMap`.
- **Dark Theme Palette:**
  - `BG = "#1E1E1E"`, `TEXT = "#EEEEEE"`, `MUTED = "#99B2B9"`, `ACC = "#00E5FF"`, `GRID = "#455A64"`
- **Motion Palette Dict:** `STATIONARY: #546E7A`, `WALKING: #26A69A`, `RUNNING: #FFA726`, `VEHICLE: #EF5350`, `UNKNOWN: #90A4AE`.
- **Cluster Palette List:** Provide a list of 10 distinct hex colors for cluster mapping (e.g., cyan, orange, green, red, purple, etc.).
- **Data Prep:** Load the three CSVs, convert `timestamp` to datetime, and sort chronologically.
- **Math/Helpers:** 1. Write a `haversine_m(lat1, lon1, lat2, lon2)` function.
  2. Write a `style_ax(ax, title, xlabel, ylabel)` helper function to apply the dark theme background, muted ticks, and grid colors to any matplotlib axes to prevent repetitive code.

🧮 2. Task 1: Evaluation Metrics

Before plotting, the LLM needs to calculate the global KPIs using a filtered dataframe that ignores "NOISE" clusters.

- **Calculations:**
  - Recalculate `pred_error_m` using the Haversine helper just in case it's missing.
  - **Accuracy:** Percentage where `cluster_label == predicted_cluster`.
  - **Errors:** Calculate the Mean, Median, 90th percentile, and 95th percentile of `pred_error_m`.
  - **Radius checks:** Percentage of predictions within 100m and 300m.
  - **Confidence:** Mean of `confidence_score`.
- Store these in a dictionary to print to the console and display on the PDF cover.

📑 3. Task 2: The 8-Page PDF Report

Instruct the LLM to use `with PdfPages(OUTPUT_PDF) as pdf:` and generate 8 specific figures (`figsize=(12, 6)` or `(12, 9)`), applying the `style_ax` helper to each.

- **Page 1: Cover Sheet**
  - Hide the axes (`ax.axis("off")`).
  - Use `ax.text()` to draw a stylized title, dataset info (dates, rows, cluster count), and a two-column table of the Evaluation Metrics calculated in Task 1. Add horizontal divider lines.
- **Page 2: Activity Distribution (1x2 Subplots)**
  - Left: A `plt.pie()` chart showing the time-share percentage of motion states.

- Right: A `plt.barh()` horizontal bar chart showing the exact percentages. Color both using `MOTION_PALETTE`.

- **Page 3: Hourly Patterns (2x1 Subplots)**
  - Top: A stacked bar chart grouping data by `hour_of_day` and `motion_state`.
  - Bottom: A "Dominant State" color strip. Calculate the most frequent motion state per hour, and draw a bar chart where all heights equal 1, colored by the dominant state. Create a custom legend using `matplotlib.patches.Patch`.
- **Page 4: Distance & Speed (2x1 Subplots)**
  - Top: A bar chart of total `distance_from_previous` grouped by `dt.date`.
  - Bottom: A line chart showing a noise-reduced speed profile using `df["speed_kmh"].rolling(8, min_periods=1).mean()`. Use `fill_between` for a shaded area under the curve.
- **Page 5: Static Cluster Map (Scatter Plot)**
  - Plot Longitude vs Latitude.
  - Plot NOISE points in dark grey (`alpha=0.4`, `zorder=1`).
  - Plot valid clusters colored by `CLUSTER_PALETTE` (`zorder=2`).
  - Loop through the cluster info dataframe and plot a large white star (`*`) at the `center_lat` / `center_lon` with text annotations for the label (`zorder=5`).
- **Page 6: Error Analysis (1x2 Subplots)**
  - Left: A histogram of `pred_error_m` with vertical dashed lines marking the Mean and Median.
  - Right: A Cumulative Distribution Function (CDF) line plot using `np.sort()`. Add vertical dashed lines at 100m and 300m.
- **Page 7: Confidence Analysis (1x2 Subplots)**
  - Left: A histogram of the `confidence_score` with a vertical dashed line for the Mean.
  - Right: A scatter plot of `confidence_score` (X) vs `pred_error_m` (Y), colored using `cmap="YlOrRd"` with a colorbar.
- **Page 8: Predictions & Transitions (1x2 Subplots)**
  - Left: A bar chart of `predicted_cluster` value counts.
  - Right: A transition matrix. Use `pd.crosstab(prev_cluster, current_cluster, normalize='index')`. Pass this matrix into `sns.heatmap(cmap="Blues", annot=True)`.

🗺️ 4. Task 3: Interactive Folium Map

Instruct the LLM to wrap Folium generation in a `try/except` block to skip it if the library isn't installed. Build a `folium.Map` using `tiles="CartoDB dark_matter"`.

Add 4 specific `folium.FeatureGroup` layers:

- **Layer 1: GPS Trail**
  - A `folium.PolyLine` connecting all historical latitude/longitude points.
  - A `folium.CircleMarker` for every point, filled with the specific `MOTION_PALETTE` color. Add a popup showing Speed and Motion State.
- **Layer 2: Cluster Zones**
  - Loop through `cluster_info`.
  - Draw a dashed `folium.Circle` using the actual `radius_m` from the file.
  - Add a `folium.Marker` at the centroid using a custom `folium.DivIcon` to render HTML text (e.g., `★ HOME`) directly on the map.
- **Layer 3: Predicted Search Zones**
  - *Crucial Logic:* Sample the dataframe (e.g., `df_pred.iloc[::SAMPLE_N]`) to prevent browser lag.
  - Draw a large translucent `folium.Circle` representing the `search_radius_m`.
  - Draw a small solid `folium.CircleMarker` at the exact `predicted_lat/lon`. Add popups showing confidence scores.
- **Layer 4: Density Heatmap**
  - Pass a list of valid coordinates into `folium.plugins.HeatMap` with a custom color gradient (blue to red).

Final Map Additions:

- Add `folium.LayerControl()` so the user can toggle the 4 layers.
- Create a fixed floating HTML `<div style="position:fixed; ...">` at the bottom-left of the screen to serve as a custom legend mapping colors to Motion States and Cluster labels. Save to `OUTPUT_MAP`.